

FrugalNav

Demos & Operations Guide

An uncertainty-scheduled VIO + AVL navigator for GPS-denied UAVs, built for ultra-low-power RISC-V.

1 The three demos at a glance

FrugalNav ships in three demo families of increasing realism. The core algorithm — a scheduler that spends an absolute landmark fix only when a confidence metric says it is needed — is identical in all three; what changes is how much the navigator is **told** versus how much it **measures**.

Demo	Motion (VIO)	Position (AVL)	Obstacles	Wind
1 — Sandbox (demo, canopy)	injected	injected	known map	injected (you set it)
2 — Real camera (real, real_dense)	simulated drift	REAL ArUco (camera)	known map	estimated
3 — Full realism (real, real_dense)	REAL optical-flow (camera)	REAL ArUco (camera)	REAL laser sensor	estimated

In Demo 3 the navigator is handed **nothing**: it estimates its own motion from the camera, fixes its position from real tag detections, detects obstacles with a real sensor, and infers the wind. Ground truth is used only to draw the error and to teleport on reset.

2 How it works

2.1 The shared idea — frugal correction

Vision-inertial odometry (VIO) tells the drone how it is moving, but that estimate **drifts**. An absolute fix from a known landmark (ArUco tag) removes the drift, but every fix costs compute and power. FrugalNav's contribution is an **uncertainty scheduler**: it tracks a confidence metric U and spends a landmark fix only when U crosses a threshold — the same accuracy as correcting on every marker, for roughly half the corrections. Fewer corrections = less compute = fits a low-power RISC-V budget.

2.2 Demo 1 — Sandbox

Two maps (**demo**, and the dense **canopy** forest) with a full weather simulator (wind, fog, rain, visibility) and keyboard flight. Signals are injected, so it is the easiest to fly and best for showing the scheduler and weather effects.

2.3 Demo 2 — Real camera perception

The drone flies on a genuine downward **camera**: a perception node runs real OpenCV ArUco detection + solvePnP on the rendered image to recover an absolute position fix (about 0.05 m after a one-time mounting calibration). The navigator is 'blind' — it sees only the fixes and image-quality cues — and **estimates the wind** by comparing the velocity it commanded to the velocity it achieved. Obstacle positions are still taken from the map, and the VIO drift is a simulated model.

2.4 Demo 3 — Full realism ("knows nothing")

Everything the navigator acts on is measured live:

- **Motion (real VIO)**: a node tracks sparse features on the downward camera with Lucas-Kanade optical flow and converts the pixel flow to a metric velocity using altitude — genuine visual odometry that drifts on its own. The camera-to-world mapping is fixed once with an orthogonal (rotation) calibration.
- **Position (real AVL)**: the same real ArUco fixes as Demo 2, spent frugally by the scheduler.

- **Obstacles (real sensor):** a 360° laser rangefinder. Detection = the nearest range in the scan; a potential field pushes the velocity away, with a goal-side swirl so it rounds pillars instead of stalling. No obstacle map.
- **Wind:** inferred from measured-minus-commanded velocity; the navigator is never told the value.

Verified end-to-end: VIO calibrates, drift stays bounded (about 2 m, corrected to ~0.3 m by just 2 fixes), obstacles are cleared on laser alone, and the drone reaches the target — all with zero privileged information.

3 Keeping it cheap (the RISC-V constraint)

The whole project targets ultra-low-power compute, so the Demo 3 sensors were chosen for a tiny CPU budget:

- **Obstacles → laser rangefinder, not camera vision.** A range sensor returns distances directly, so detection is a handful of comparisons per cycle — the class of thing a sub-\$5 ToF sensor does on an 8-bit micro. Camera-based obstacle detection (stereo depth, dense optical-flow time-to-contact, or a neural net) is exactly the CPU-heavy work that would blow the power budget, so it is deliberately avoided.
- **Motion → sparse optical flow.** Tracking ~90 features with Lucas-Kanade is the same computation as the PX4Flow sensor, which runs on a small MCU — no dense flow, no learning.
- **The forward camera is for the human, not the loop.** It only feeds the on-screen view, so it costs the navigator nothing.

4 Running the demos

Everything runs in WSL Ubuntu 22.04. Open a fresh Ubuntu terminal per “Terminal N”. **Before every run**, clean up leftovers and start exactly one launch:

```
kill -9 -f gzserver; kill -9 -f gzclient; kill -9 -f rviz2; kill -9 -f frugalnav; sleep 3
```

Begin **every** terminal with these three source lines:

```
source /opt/ros/humble/setup.bash
source /usr/share/gazebo/setup.sh
source /mnt/c/Users/parth/Downloads/drone/ros2_ws/install/setup.bash
```

4.1 Demo 1 — Sandbox (demo / canopy)

```
# Terminal 1 - launch the world (use map:=canopy for the forest)
ros2 launch frugalnav_ros interactive_demo.launch.py map:=demo
```

```
# Terminal 2 - fly it (WASD, 2>manual, R=reset, weather keys)
ros2 run frugalnav_ros frugalnav_teleop.py
```

4.2 Demo 2 — Real camera (real / real_dense)

```
# Terminal 1 - launch, held for mission control (add map:=real_dense for the cluttered arena)
ros2 launch frugalnav_ros real_demo.launch.py map:=real start_paused:=true gui:=true
```

```
# Terminal 2 - mission control (SPACE=play/pause, R=reset, I/J/K/L=place, ]/[=wind)
ros2 run frugalnav_ros frugalnav_mission_control.py
```

```
# Terminal 3 - the drone's camera feed
ros2 run rqt_image_view rqt_image_view /frugalnav/down_cam/annotated
```

4.3 Demo 3 — Full realism (real / real_dense)

```
# Terminal 1 - launch, held (add map:=real_dense for the cluttered arena)
ros2 launch frugalnav_ros real3_demo.launch.py map:=real start_paused:=true gui:=true
```

```
# Terminal 2 - mission control (same keys as Demo 2)
ros2 run frugalnav_ros frugalnav_mission_control.py
```

```
# Terminal 3 - downward feed (what AVL + VIO see)
ros2 run rqt_image_view rqt_image_view /frugalnav/down_cam/annotated
```

```
# Terminal 4 - FRONT camera feed (obstacle view)
ros2 run rqt_image_view rqt_image_view /frugalnav/front_cam/annotated
```

Drop `start_paused:=true` to fly immediately; drop `gui:=true` to skip the heavy Gazebo 3D window and watch in RViz only. If a window shows only as a taskbar icon, click it (a WSLg quirk); the real demos need only RViz + the feeds, not the Gazebo window.

5 Viewing the camera feeds

What you want to see	Demo	Command
Downward feed (ArUco detection drawn + fix error)	2 and 3	<code>ros2 run rqt_image_view rqt_image_view /frugalnav/down_cam/annotated</code>
Front / approach camera (laser proximity overlay)	3 only	<code>ros2 run rqt_image_view rqt_image_view /frugalnav/front_cam/annotated</code>
Raw (no overlay)	2 and 3	<code>.../down_cam/image_raw</code> or <code>.../front_cam/image_raw</code>

The downward window shows the tags outlined with a live FIX (x, y) err= readout — the drone seeing and locking onto landmarks. The front window (Demo 3) shows **OBSTACLE AHEAD: n.n m** in amber, a red border + **AVOIDING** when a pillar is close, and **PATH CLEAR** otherwise — all driven by the laser, so the feed costs the navigator nothing.

6 Mission-control keys (Demos 2 & 3)

Key	Action	Key	Action
SPACE	play / pause (a true freeze)	] / [	wind stronger / weaker
R	reset - teleport home & hold	T	rain / gust on-off
I / K / J / L	place the drone (N/S/W/E)	G	weather on-off
1 / 2	AUTO / MANUAL	W A S D	fly (manual); X = stop

Typical flow: launch held → nudge the start point with I/J/K/L → set the weather → SPACE to fly → R any time to rewind to that start.

7 What is real, and honest scope

- **Real in Demo 3:** the VIO (optical-flow odometry from images), the AVL (ArUco detection), the obstacle sensing (laser), and the wind inference. None of these are handed to the navigator.
- **Scoped / simulated:** the arena and sensors are simulated in Gazebo; the RISC-V target is a feasibility study (C++ port + profiling), not silicon. Demos 1-2 keep a simulated VIO drift and a known obstacle map by design — Demo 3 is the one that removes both.
- **Behaviour:** the laser avoider is a reactive potential field, so it takes a conservative, sometimes curved path around obstacles rather than an optimal one — expected for a cheap reactive method.

FrugalNav — generated operations guide. Full command reference: docs/RUN_COMMANDS.md in the repository.